

3ª LISTA DE EXERCÍCIOS E TRABALHOS PRÁTICOS DE COMPUTAÇÃO GRÁFICA

Entrega: 26/06/2026 no Classroom da disciplina.

Entregar as questões respondidas, o código fonte e saídas gráficas dos programas.

Questões sobre Polinômio Cúbico de Hermite, Curvas de Bézier, Algoritmo de De Casteljau e B-Splines

1. Explique a diferença entre uma curva definida por **interpolação** e uma curva definida por **aproximação**. Em qual dessas categorias se enquadram, em geral, as curvas de Hermite, as curvas de Bézier e as B-Splines?
2. Considere um polinômio cúbico paramétrico escrito na forma:

$$P(t) = at^3 + bt^2 + ct + d, \quad 0 \leq t \leq 1.$$

Explique por que são necessárias **quatro condições independentes** para determinar completamente esse polinômio.

3. No caso do **polinômio cúbico de Hermite**, quais são as quatro condições geométricas normalmente utilizadas para determinar a curva entre dois pontos?
4. Uma curva cúbica de Hermite é definida pelos pontos extremos P_0 e P_1 , e pelos vetores tangentes T_0 e T_1 . Explique o papel geométrico de cada um desses elementos na forma final da curva.
5. Deduza a forma matricial da curva cúbica de Hermite a partir das condições:

$$P(0) = P_0, \quad P(1) = P_1, \quad P'(0) = T_0, \quad P'(1) = T_1.$$

6. Explique o significado das funções de base de Hermite. Por que elas podem ser interpretadas como pesos associados aos pontos extremos e aos vetores tangentes?
7. Compare a curva cúbica de Hermite com a curva cúbica de Bézier. Quais elementos controlam a tangente inicial e a tangente final em cada caso?
8. Uma curva de Bézier cúbica é definida por quatro pontos de controle:

$$P_0, P_1, P_2, P_3.$$

Explique por que a curva passa necessariamente por P_0 e P_3 , mas não necessariamente por P_1 e P_2 .

9. Dada a curva de Bézier cúbica:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3,$$

mostre que:

$$B(0) = P_0$$

e

$$B(1) = P_3.$$

10. Para uma curva de Bézier cúbica, mostre que as derivadas nos extremos são dadas por:

$$B'(0) = 3(P_1 - P_0)$$

e

$$B'(1) = 3(P_3 - P_2).$$

Explique o significado geométrico dessas expressões.

11. Considere uma curva de Bézier cúbica com pontos de controle:

$$P_0 = (0, 0), \quad P_1 = (1, 3), \quad P_2 = (3, 3), \quad P_3 = (4, 0).$$

Determine as equações paramétricas $x(t)$ e $y(t)$.

12. Explique a propriedade do **fecho convexo** nas curvas de Bézier. Por que essa propriedade é importante em Computação Gráfica?
13. O que significa dizer que as curvas de Bézier possuem **invariância afim**? Dê um exemplo envolvendo uma transformação geométrica, como translação, rotação ou escala.
14. Descreva o **Algoritmo de De Casteljau** para uma curva de Bézier quadrática. Mostre como o ponto da curva é obtido por interpolações lineares sucessivas.
15. Aplique o Algoritmo de De Casteljau a uma curva de Bézier cúbica. Indique claramente as interpolações de primeiro, segundo e terceiro nível.
16. Explique por que o Algoritmo de De Casteljau é considerado geometricamente intuitivo e numericamente estável para o cálculo de pontos em curvas de Bézier.
17. Compare o cálculo direto da curva de Bézier usando polinômios de Bernstein com o cálculo usando o Algoritmo de De Casteljau. Quais são as vantagens e desvantagens de cada abordagem?
18. Explique o conceito de **B-Spline**. Em que sentido uma B-Spline pode ser vista como uma generalização das curvas de Bézier?
19. Qual é a função do **vetor de nós** em uma B-Spline? Explique como a distribuição dos nós influencia o comportamento da curva.
20. Compare curvas de Bézier e B-Splines quanto aos seguintes aspectos: controle local, interpolação dos pontos extremos, influência dos pontos de controle, continuidade entre segmentos e aplicação em modelagem geométrica.

Projetos Práticos sobre Curvas de Bézier

Projeto 1: Editor Interativo de Curvas de Bézier

Objetivo:

Desenvolver uma aplicação gráfica que permita ao usuário criar, visualizar e modificar curvas de Bézier por meio da manipulação direta dos pontos de controle.

Descrição:

O aluno deverá implementar um editor simples no qual seja possível inserir pontos de controle com o mouse e desenhar a curva de Bézier correspondente. A aplicação deve permitir mover os pontos de controle e atualizar a curva em tempo real.

Conceitos explorados:

- representação paramétrica de curvas;
- pontos de controle;
- polinômios de Bernstein;
- curvas de Bézier quadráticas e cúbicas;
- interação com mouse;
- atualização dinâmica da cena.

Atividades sugeridas:

1. Criar uma janela gráfica usando OpenGL, Python, JavaScript Canvas ou outra biblioteca gráfica.
2. Permitir a inserção de pontos de controle.
3. Desenhar o polígono de controle.
4. Calcular os pontos da curva para $0 \leq t \leq 1$.
5. Exibir a curva suavemente na tela.
6. Permitir que o usuário selecione e mova pontos de controle.
7. Comparar o comportamento de curvas quadráticas, cúbicas e de grau superior.

Produto final:

Um editor visual no qual o usuário possa construir curvas de Bézier interativamente e observar como os pontos de controle alteram a forma da curva.

Projeto 2: Animação de um Objeto ao Longo de uma Curva de Bézier**Objetivo:**

Criar uma animação em que um objeto se desloca suavemente ao longo de uma trajetória definida por uma curva de Bézier.

Descrição:

O projeto consiste em definir uma curva de Bézier cúbica e utilizar seus pontos para controlar o movimento de um objeto na tela, como um círculo.

Conceitos explorados:

- curvas paramétricas;
- animação baseada no parâmetro t ;

- interpolação de posição;
- controle de velocidade;
- orientação do objeto pela tangente da curva;
- derivada da curva de Bézier.

Atividades sugeridas:

1. Definir quatro pontos de controle P_0, P_1, P_2, P_3 .
2. Implementar a equação da curva de Bézier cúbica:

$$B(t) = (1 - t)^3 P_0 + 3(1 - t)^2 t P_1 + 3(1 - t) t^2 P_2 + t^3 P_3.$$

3. Fazer o parâmetro t variar de 0 a 1.
4. Desenhar o objeto na posição $B(t)$.
5. Calcular a tangente da curva para orientar o objeto.
6. Ajustar a velocidade da animação.
7. Reiniciar ou inverter o movimento ao final da curva.

Produto final:

Uma animação na qual um objeto percorre uma trajetória suave definida por uma curva de Bézier.

Projeto 3: Visualização do Algoritmo de De Casteljau

Objetivo:

Implementar uma ferramenta didática que mostre passo a passo o funcionamento do Algoritmo de De Casteljau para curvas de Bézier de grau n .

Descrição:

O projeto deve permitir visualizar as interpolações lineares sucessivas que geram um ponto da curva de Bézier. O parâmetro t deve ser controlado por teclado, mouse ou barra deslizante.

Conceitos explorados:

- interpolação linear;
- construção geométrica de curvas;
- Algoritmo de De Casteljau;
- estabilidade numérica;
- visualização didática;
- subdivisão de curvas.

Atividades sugeridas:

1. Definir pontos de controle para uma curva quadrática ou cúbica.

2. Desenhar o polígono de controle inicial.
3. Para cada valor de t , calcular os pontos intermediários por interpolação linear:

$$Q_i(t) = (1 - t)P_i + tP_{i+1}.$$
4. Repetir o processo até obter o ponto final da curva.
5. Exibir graficamente cada nível de interpolação.
6. Permitir variar t dinamicamente.
7. Mostrar o ponto correspondente da curva sendo formado.

Produto final:

Uma visualização interativa do Algoritmo de De Casteljau, mostrando como a curva é construída geometricamente.

Projeto 4: Modelagem de Pistas, Estradas ou Rotas com Curvas de Bézier

O traçado geométrico de uma estrada de rodagem consiste na definição espacial do eixo da via, considerando aspectos de segurança, conforto, velocidade, topografia, economia de construção e integração com o ambiente. Em termos geométricos, uma estrada pode ser entendida como uma curva no espaço, normalmente projetada a partir de três componentes principais: o alinhamento horizontal, o alinhamento vertical e a seção transversal.

O alinhamento horizontal define a projeção da estrada sobre o plano, isto é, como a via se desenvolve quando vista de cima. Já o alinhamento vertical descreve o perfil longitudinal da estrada, indicando rampas, aclives, declives e curvas verticais. A seção transversal define elementos como largura da pista, acostamentos, taludes, superelevação e drenagem.

No contexto das curvas de Bézier, o maior interesse está inicialmente no alinhamento horizontal, embora elas também possam ser utilizadas na modelagem de perfis verticais e trajetórias tridimensionais.

As curvas de Bézier oferecem uma abordagem muito poderosa do ponto de vista computacional e gráfico. Elas permitem desenhar e ajustar trajetórias suaves por meio de pontos de controle, sendo especialmente úteis em ambientes CAD, simulações, jogos, visualização 3D, animações e estudos preliminares de traçado.

Objetivo:

Utilizar curvas de Bézier para modelar trajetórias suaves, como pistas de corrida, estradas, rotas de drones ou caminhos para personagens em jogos.

Descrição:

O aluno deverá criar uma cena gráfica em que uma rota é formada por uma ou mais curvas de Bézier conectadas. A rota pode representar uma estrada, uma trajetória aérea, uma linha férrea ou o caminho de um robô móvel.

Conceitos explorados:

- modelagem geométrica de trajetórias;
- conexão de segmentos de curvas;

- continuidade entre curvas;
- suavidade de caminhos;
- orientação por tangente;
- aplicação em jogos e simulações.

Atividades sugeridas:

1. Definir uma sequência de pontos de controle.
2. Criar vários segmentos de Bézier cúbicos.
3. Conectar os segmentos para formar uma rota contínua.
4. Ajustar os pontos de controle para evitar mudanças bruscas de direção.
5. Desenhar a rota completa na tela.
6. Animar um objeto percorrendo essa rota.
7. Usar a tangente da curva para orientar o objeto durante o movimento.

Produto final:

Uma aplicação gráfica que modele e visualize uma rota suave usando curvas de Bézier, podendo incluir a animação de um objeto percorrendo o caminho.

Código de Apoio

Curva Cúbica de Bézier

```
# Traçado de curva cúbica de Bezier
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import sys

# Lista de pontos de controle
control_points = []

# Tamanho da janela
width, height = 800, 600

def bezier_point(t, p0, p1, p2, p3):
    """Calcula ponto da curva de Bézier cúbica"""
    x = (1 - t)**3 * p0[0] + 3 * (1 - t)**2 * t * p1[0] +
    3 * (1 - t) * t**2 * p2[0] + t**3 * p3[0]

    y = (1 - t)**3 * p0[1] + 3 * (1 - t)**2 * t * p1[1] +
    3 * (1 - t) * t**2 * p2[1] + t**3 * p3[1]

    return (x, y)
```

```

def draw_bezier_curve():
    """Desenha a curva de Bézier"""
    glColor3f(1.0, 1.0, 0.0) # Amarelo
    glLineWidth(2.0)
    glBegin(GL_LINE_STRIP)
    for i in range(101):
        t = i / 100.0
        pt = bezier_point(t, *control_points)
        glVertex2f(pt[0], pt[1])
    glEnd()

def draw_control_points():
    """Desenha os pontos de controle"""
    glColor3f(1.0, 0.0, 0.0) # Vermelho
    glPointSize(6.0)
    glBegin(GL_POINTS)
    for pt in control_points:
        glVertex2f(pt[0], pt[1])
    glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    draw_control_points()
    if len(control_points) == 4:
        draw_bezier_curve()
    glutSwapBuffers()

def mouse_click(button, state, x, y):
    if button == GLUT_LEFT_BUTTON and state == GLUT_DOWN:
        if len(control_points) < 4:
            # Converte coordenadas da janela para coordenadas OpenGL
            gl_x = x
            gl_y = height - y
            control_points.append((gl_x, gl_y))
            glutPostRedisplay()

def reshape(w, h):
    global width, height
    width, height = w, h
    glViewport(0, 0, w, h)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluOrtho2D(0, w, 0, h)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

def main():
    glutInit(sys.argv)
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)

```

```

    glutInitWindowSize(width, height)
    glutCreateWindow("Curva de Bézier com Mouse".encode("utf-8"))
    glClearColor(0.2, 0.2, 0.2, 1.0)
    glutDisplayFunc(display)
    glutMouseFunc(mouse_click)
    glutReshapeFunc(reshape)
    glutMainLoop()

if __name__ == "__main__":
    main()

```

Curva Cúbica de Bézier Interativa

```

# Bezier com interação nos pontos de controle
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import sys

# Lista de pontos de controle
control_points = []
selected_point = None
width, height = 800, 600
radius = 8 # Raio de detecção para seleção de ponto

def bezier_point(t, p0, p1, p2, p3):
    """Calcula ponto da curva de Bézier cúbica"""
    x = (1 - t)**3 * p0[0] + 3 * (1 - t)**2 * t * p1[0] +
    3 * (1 - t) * t**2 * p2[0] + t**3 * p3[0]

    y = (1 - t)**3 * p0[1] + 3 * (1 - t)**2 * t * p1[1] +
    3 * (1 - t) * t**2 * p2[1] + t**3 * p3[1]

    return (x, y)

def draw_bezier_curve():
    """Desenha a curva de Bézier"""
    if len(control_points) == 4:
        glColor3f(1.0, 1.0, 0.0) # Amarelo
        glLineWidth(2.0)
        glBegin(GL_LINE_STRIP)
        for i in range(101):
            t = i / 100.0
            pt = bezier_point(t, *control_points)
            glVertex2f(pt[0], pt[1])
        glEnd()

def draw_control_points():
    """Desenha os pontos de controle"""

```



```

        glColor3f(1.0, 0.0, 0.0) # Vermelho
        glPointSize(10.0)
        glBegin(GL_POINTS)
        for pt in control_points:
            glVertex2f(pt[0], pt[1])
        glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    draw_control_points()
    draw_bezier_curve()
    glutSwapBuffers()

def reshape(w, h):
    global width, height
    width, height = w, h
    glViewport(0, 0, w, h)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluOrtho2D(0, w, 0, h)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

def mouse_click(button, state, x, y):
    global selected_point
    gl_x = x
    gl_y = height - y

    if button == GLUT_LEFT_BUTTON:
        if state == GLUT_DOWN:
            if len(control_points) < 4:
                control_points.append((gl_x, gl_y))
            else:
                # Verifica se clicou próximo de algum ponto
                for i, pt in enumerate(control_points):
                    dx = pt[0] - gl_x
                    dy = pt[1] - gl_y
                    if dx * dx + dy * dy <= radius * radius:
                        selected_point = i
                        break
        elif state == GLUT_UP:
            selected_point = None

    glutPostRedisplay()

def mouse_motion(x, y):
    global selected_point
    if selected_point is not None:
        gl_x = x
        gl_y = height - y

```

```

        control_points[selected_point] = (gl_x, gl_y)
        glutPostRedisplay()

def main():
    glutInit(sys.argv)
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)
    glutInitWindowSize(width, height)
    glutCreateWindow("Curva de Bézier Interativa".encode("utf-8"))
    glClearColor(0.2, 0.2, 0.2, 1.0)
    glutDisplayFunc(display)
    glutReshapeFunc(reshape)
    glutMouseFunc(mouse_click)
    glutMotionFunc(mouse_motion)
    glutMainLoop()

if __name__ == "__main__":
    main()

```